Assignment 1: Feedforward Neural Network from Scratch

Field	Details
Name	Shivam Khedkar
Batch	1
PRN	202301040063
Assignment	1
Topic	Implementing a Simple Feedforward Neural Network from Scratch in Python

---

Objective

Implement a feedforward neural network **without any deep learning libraries** (no TensorFlow, no PyTorch, no Keras).
We implement:

- Forward Pass
- Backpropagation
- Gradient Descent (Weight Update)
- Display all updated weights after every Epoch

Theory

A **Feedforward Neural Network (FNN)** is the simplest type of artificial neural network. Information moves in only one direction — forward — from input nodes, through hidden nodes, to output nodes.

Key Concepts

1. Forward Pass:
For each layer: `Z = X · W + b`, then `A = activation(Z)`

2. Loss Function (MSE):
`Loss = (1/n) * Σ(y_pred - y_true)²`

3. Backpropagation:
Compute gradients using chain rule:
`dW = (1/n) * Xᵀ · dZ`, `db = (1/n) * Σ dZ`

4. Weight Update (Gradient Descent):
`W = W - learning_rate * dW`
`b = b - learning_rate * db`

Network Architecture Used

```
Input Layer  →  Hidden Layer (Sigmoid)  →  Output Layer (Sigmoid)
   2 neurons         4 neurons                    1 neuron
```

Step 1: Import Only NumPy

```
import numpy as np
import matplotlib.pyplot as plt

# Set random seed for reproducibility
np.random.seed(42)
print("Libraries imported successfully!")
print("Note: NO deep learning libraries used (no TensorFlow / PyTorch / Kera
```

```
Libraries imported successfully!
Note: NO deep learning libraries used (no TensorFlow / PyTorch / Keras)
```

Step 2: Prepare the Dataset (XOR Problem)

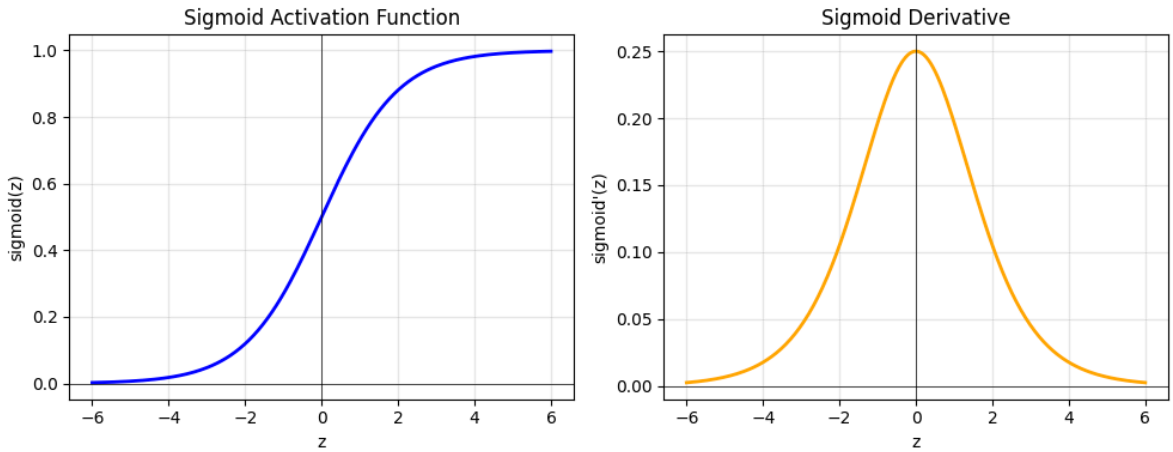
```
# XOR Dataset — classic non-linearly separable problem
# Input: 4 samples, 2 features each
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Output: XOR labels
y = np.array([[0],
              [1],
              [1],
              [0]])

print("Dataset (XOR Problem):")
print("-" * 30)
print(f"{'Input X1':<12} {'Input X2':<12} {'Output Y'}")
print("-" * 30)
for i in range(len(X)):
    print(f"{X[i][0]:<12} {X[i][1]:<12} {y[i][0]}")
print("-" * 30)
print(f"\nX shape: {X.shape}")
print(f"y shape: {y.shape}")
```

```
Dataset (XOR Problem):
------------------------------
Input X1     Input X2     Output Y
------------------------------
0            0            0
0            1            1
```

```
1            0            1
1            1            0
------------------------------

X shape: (4, 2)
y shape: (4, 1)
```

Step 3: Define Activation Functions

```
def sigmoid(z):
    """Sigmoid activation function: f(z) = 1 / (1 + e^(-z))"""
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    """Derivative of sigmoid: f'(z) = f(z) * (1 - f(z))"""
    s = sigmoid(z)
    return s * (1 - s)

def mse_loss(y_true, y_pred):
    """Mean Squared Error Loss"""
    return np.mean((y_true - y_pred) ** 2)

# Visualize sigmoid
z_vals = np.linspace(-6, 6, 200)
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(z_vals, sigmoid(z_vals), color='blue', linewidth=2)
plt.title('Sigmoid Activation Function')
plt.xlabel('z'); plt.ylabel('sigmoid(z)')
plt.grid(True, alpha=0.3); plt.axhline(0, color='k', lw=0.5); plt.axvline(0,

plt.subplot(1, 2, 2)
plt.plot(z_vals, sigmoid_derivative(z_vals), color='orange', linewidth=2)
plt.title('Sigmoid Derivative')
plt.xlabel('z'); plt.ylabel("sigmoid'(z)")
plt.grid(True, alpha=0.3); plt.axhline(0, color='k', lw=0.5); plt.axvline(0,

plt.tight_layout()
plt.show()
print("Activation functions defined!")
```




```
Activation functions defined!
```

Step 4: Initialize Network Weights and Biases

```
# Network Architecture
input_size  = 2   # 2 input features
hidden_size = 4   # 4 neurons in hidden layer
output_size = 1   # 1 output neuron

# ---- Initialize Weights (small random values) ----
W1 = np.random.randn(input_size, hidden_size) * 0.5   # Shape: (2, 4)
b1 = np.zeros((1, hidden_size))                        # Shape: (1, 4)

W2 = np.random.randn(hidden_size, output_size) * 0.5  # Shape: (4, 1)
b2 = np.zeros((1, output_size))                        # Shape: (1, 1)

print("=" * 50)
print("   NETWORK ARCHITECTURE & INITIAL WEIGHTS")
print("=" * 50)
print(f"Input  Layer : {input_size} neurons")
print(f"Hidden Layer : {hidden_size} neurons  (Sigmoid activation)")
print(f"Output Layer : {output_size} neuron   (Sigmoid activation)")
print()
print(f"W1 (Input→Hidden) shape : {W1.shape}")
print(f"W1 initial values:\n{W1}")
print()
print(f"b1 (Hidden bias) shape  : {b1.shape}")
print(f"b1 initial values:\n{b1}")
print()
print(f"W2 (Hidden→Output) shape: {W2.shape}")
```

```
print(f"W2 initial values:\n{W2}")
print()
print(f"b2 (Output bias) shape  : {b2.shape}")
print(f"b2 initial values:\n{b2}")
print("=" * 50)
```

```
==================================================
   NETWORK ARCHITECTURE & INITIAL WEIGHTS
==================================================
Input  Layer : 2 neurons
Hidden Layer : 4 neurons  (Sigmoid activation)
Output Layer : 1 neuron   (Sigmoid activation)

W1 (Input→Hidden) shape : (2, 4)
W1 initial values:
[[ 0.24835708 -0.06913215  0.32384427  0.76151493]
 [-0.11707669 -0.11706848  0.78960641  0.38371736]]

b1 (Hidden bias) shape  : (1, 4)
b1 initial values:
[[0. 0. 0. 0.]]

W2 (Hidden→Output) shape: (4, 1)
W2 initial values:
[[-0.23473719]
 [ 0.27128002]
 [-0.23170885]
 [-0.23286488]]

b2 (Output bias) shape  : (1, 1)
b2 initial values:
[[0.]]
==================================================
```

Step 5: Forward Pass

```
def forward_pass(X, W1, b1, W2, b2):
    """
    Perform forward pass through the network.

    Layer 1 (Hidden):
        Z1 = X · W1 + b1
        A1 = sigmoid(Z1)

    Layer 2 (Output):
        Z2 = A1 · W2 + b2
        A2 = sigmoid(Z2)  ← final prediction
    """
    # Hidden Layer
    Z1 = np.dot(X, W1) + b1       # Linear combination
    A1 = sigmoid(Z1)               # Activation

    # Output Layer
    Z2 = np.dot(A1, W2) + b2      # Linear combination
    A2 = sigmoid(Z2)               # Final prediction
```

```
    # Cache for backprop
    cache = {'Z1': Z1, 'A1': A1, 'Z2': Z2, 'A2': A2}
    return A2, cache

# Test forward pass with initial weights
y_pred, cache = forward_pass(X, W1, b1, W2, b2)
print("Forward Pass Test (before training):")
print("-" * 40)
print(f"{'Input':<15} {'Predicted':<15} {'Actual'}")
print("-" * 40)
for i in range(len(X)):
    print(f"{str(X[i]):<15} {y_pred[i][0]:<15.6f} {y[i][0]}")
print("-" * 40)
print(f"Initial Loss (MSE): {mse_loss(y, y_pred):.6f}")
```

```
Forward Pass Test (before training):
----------------------------------------
Input           Predicted       Actual
----------------------------------------
[0 0]           0.446699        0
[0 1]           0.430293        1
[1 0]           0.426999        1
[1 1]           0.412631        0
----------------------------------------
Initial Loss (MSE): 0.255675
```

Step 6: Backpropagation

```
def backward_pass(X, y, cache, W1, W2):
    """
    Backpropagation: Compute gradients using chain rule.

    Output Layer gradients:
        dLoss/dA2 = 2*(A2 - y) / n
        dA2/dZ2   = sigmoid'(Z2)
        dZ2       = dLoss/dA2 * dA2/dZ2
        dW2       = A1ᵀ · dZ2
        db2       = sum(dZ2)

    Hidden Layer gradients:
        dA1       = dZ2 · W2ᵀ
        dZ1       = dA1 * sigmoid'(Z1)
        dW1       = Xᵀ · dZ1
        db1       = sum(dZ1)
    """
    n = X.shape[0]
    Z1, A1, Z2, A2 = cache['Z1'], cache['A1'], cache['Z2'], cache['A2']

    # ── Output Layer ──────────────────────────────
    dA2 = 2 * (A2 - y) / n
    dZ2 = dA2 * sigmoid_derivative(Z2)
    dW2 = np.dot(A1.T, dZ2)
```

```
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    # ── Hidden Layer ──────────────────────────────
    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * sigmoid_derivative(Z1)
    dW1 = np.dot(X.T, dZ1)
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    gradients = {'dW1': dW1, 'db1': db1, 'dW2': dW2, 'db2': db2}
    return gradients

print("Backward pass function defined successfully!")
print()

# Test backward pass
grads = backward_pass(X, y, cache, W1, W2)
print("Sample Gradients (before training):")
print(f"dW1 shape: {grads['dW1'].shape}\n{grads['dW1']}")
print(f"\ndW2 shape: {grads['dW2'].shape}\n{grads['dW2']}")
```

```
Backward pass function defined successfully!

Sample Gradients (before training):
dW1 shape: (2, 4)
[[ 0.00112904 -0.00138634  0.00179967  0.00140993]
 [ 0.00116202 -0.00135758  0.00131838  0.00178703]]

dW2 shape: (4, 1)
[[-0.01801012]
 [-0.01642889]
 [-0.02345704]
 [-0.02378061]]
```

Step 7: Weight Update (Gradient Descent)

```
def update_weights(W1, b1, W2, b2, gradients, learning_rate):
    """
    Gradient Descent weight update rule:
        W = W - lr * dW
        b = b - lr * db
    """
    W1 = W1 - learning_rate * gradients['dW1']
    b1 = b1 - learning_rate * gradients['db1']
    W2 = W2 - learning_rate * gradients['dW2']
    b2 = b2 - learning_rate * gradients['db2']
    return W1, b1, W2, b2

print("Weight update function defined!")
print("Update Rule: W_new = W_old - learning_rate × gradient")
```

```
Weight update function defined!
Update Rule: W_new = W_old - learning_rate × gradient
```

Step 8: Training Loop — Display All Weights Every Epoch

```
# ── Hyperparameters ──────────────────────────────────────
learning_rate = 0.5
epochs        = 10000
print_every   = 1000   # Print weights every N epochs

# Re-initialize fresh weights
np.random.seed(42)
W1 = np.random.randn(input_size, hidden_size) * 0.5
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size) * 0.5
b2 = np.zeros((1, output_size))

loss_history = []

print("=" * 70)
print(f"   TRAINING STARTED | LR={learning_rate} | Epochs={epochs}")
print("=" * 70)

for epoch in range(1, epochs + 1):

    # ── Forward Pass ──
    y_pred, cache = forward_pass(X, W1, b1, W2, b2)

    # ── Compute Loss ──
    loss = mse_loss(y, y_pred)
    loss_history.append(loss)

    # ── Backward Pass ──
    gradients = backward_pass(X, y, cache, W1, W2)

    # ── Update Weights ──
    W1, b1, W2, b2 = update_weights(W1, b1, W2, b2, gradients, learning_rate

    # ── Print Weights Every `print_every` Epochs ──
    if epoch % print_every == 0 or epoch == 1:
        print(f"\n{'─'*70}")
        print(f"  EPOCH {epoch:>6}  |  Loss (MSE): {loss:.8f}")
        print(f"{'─'*70}")

        print(f"\n  W1 [Input → Hidden]  shape={W1.shape}")
        print(f"  {'Neuron H1':>12} {'Neuron H2':>12} {'Neuron H3':>12} {'Ne
        for r in range(W1.shape[0]):
            row_label = f"  Input {r+1}:"
            row_vals  = "  ".join(f"{W1[r,c]:>10.6f}" for c in range(W1.shap
            print(f"{row_label:<12} {row_vals}")

        print(f"\n  b1 [Hidden Bias]:")
        print("  " + "  ".join(f"{b1[0,c]:>10.6f}" for c in range(b1.shape[1

        print(f"\n  W2 [Hidden → Output]  shape={W2.shape}")
        for r in range(W2.shape[0]):
```

```
            print(f"  Hidden {r+1} → Out: {W2[r,0]:>10.6f}")

        print(f"\n  b2 [Output Bias]: {b2[0,0]:>10.6f}")

print(f"\n{'='*70}")
print("   TRAINING COMPLETE")
print(f"{'='*70}")
```

```
======================================================================
   TRAINING STARTED | LR=0.5 | Epochs=10000
======================================================================

──────────────────────────────────────────────────────────────────────
  EPOCH      1  |  Loss (MSE): 0.25567530
──────────────────────────────────────────────────────────────────────

  W1 [Input → Hidden]  shape=(2, 4)
     Neuron H1    Neuron H2    Neuron H3    Neuron H4
  Input 1:     0.247793   -0.068439    0.322944    0.760810
  Input 2:    -0.117658   -0.116390    0.788947    0.382824

  b1 [Hidden Bias]:
   -0.000987    0.001181   -0.001038   -0.001058

  W2 [Hidden → Output]  shape=(4, 1)
  Hidden 1 → Out:  -0.225732
  Hidden 2 → Out:   0.279494
  Hidden 3 → Out:  -0.219980
  Hidden 4 → Out:  -0.220975

  b2 [Output Bias]:   0.017360

──────────────────────────────────────────────────────────────────────
  EPOCH   1000  |  Loss (MSE): 0.24966092
──────────────────────────────────────────────────────────────────────

  W1 [Input → Hidden]  shape=(2, 4)
     Neuron H1    Neuron H2    Neuron H3    Neuron H4
  Input 1:     0.264884   -0.045890    0.276529    0.794192
  Input 2:    -0.115511   -0.110537    0.764381    0.440620

  b1 [Hidden Bias]:
    0.035209   -0.001709   -0.018008    0.023778

  W2 [Hidden → Output]  shape=(4, 1)
  Hidden 1 → Out:  -0.294286
  Hidden 2 → Out:   0.208350
  Hidden 3 → Out:   0.061019
  Hidden 4 → Out:   0.218046

  b2 [Output Bias]:  -0.119853

──────────────────────────────────────────────────────────────────────
  EPOCH   2000  |  Loss (MSE): 0.20300835
──────────────────────────────────────────────────────────────────────

  W1 [Input → Hidden]  shape=(2, 4)
     Neuron H1    Neuron H2    Neuron H3    Neuron H4
  Input 1:     0.670996   -0.070602    0.282029    2.658598
```

```
  Input 2:     0.119156   -0.145111    0.845543    2.595437

  b1 [Hidden Bias]:
    0.123987   -0.012318   -0.070447   -0.219932

  W2 [Hidden → Output]  shape=(4, 1)
```

Step 9: Final Predictions & Evaluation

```
y_pred_final, _ = forward_pass(X, W1, b1, W2, b2)
y_pred_class    = (y_pred_final >= 0.5).astype(int)

print("=" * 55)
print("         FINAL PREDICTIONS AFTER TRAINING")
print("=" * 55)
print(f"{'Input':<12} {'Raw Output':<15} {'Predicted':<12} {'Actual':<8} {'C
print("-" * 55)
correct = 0
for i in range(len(X)):
    pred = y_pred_class[i][0]
    act  = y[i][0]
    ok   = "✓" if pred == act else "✗"
    if pred == act:
        correct += 1
    print(f"{str(X[i]):<12} {y_pred_final[i][0]:<15.6f} {pred:<12} {act:<8}
print("-" * 55)
print(f"Final Loss (MSE) : {mse_loss(y, y_pred_final):.8f}")
print(f"Accuracy         : {correct}/{len(X)} = {correct/len(X)*100:.1f}%")
print("=" * 55)
```

```
=======================================================
         FINAL PREDICTIONS AFTER TRAINING
=======================================================
Input        Raw Output      Predicted    Actual   Correct?
-------------------------------------------------------
[0 0]        0.029803        0            0        ✓
[0 1]        0.974839        1            1        ✓
[1 0]        0.974751        1            1        ✓
[1 1]        0.025032        0            0        ✓
-------------------------------------------------------
Final Loss (MSE) : 0.00069635
Accuracy         : 4/4 = 100.0%
=======================================================
```

Step 10: Display All Final Weights

```
print("=" * 60)
print("         FINAL LEARNED WEIGHTS & BIASES")
print("=" * 60)

print("\n[ W1 ] Input → Hidden Layer  (shape: 2×4)")
print("-" * 55)
```

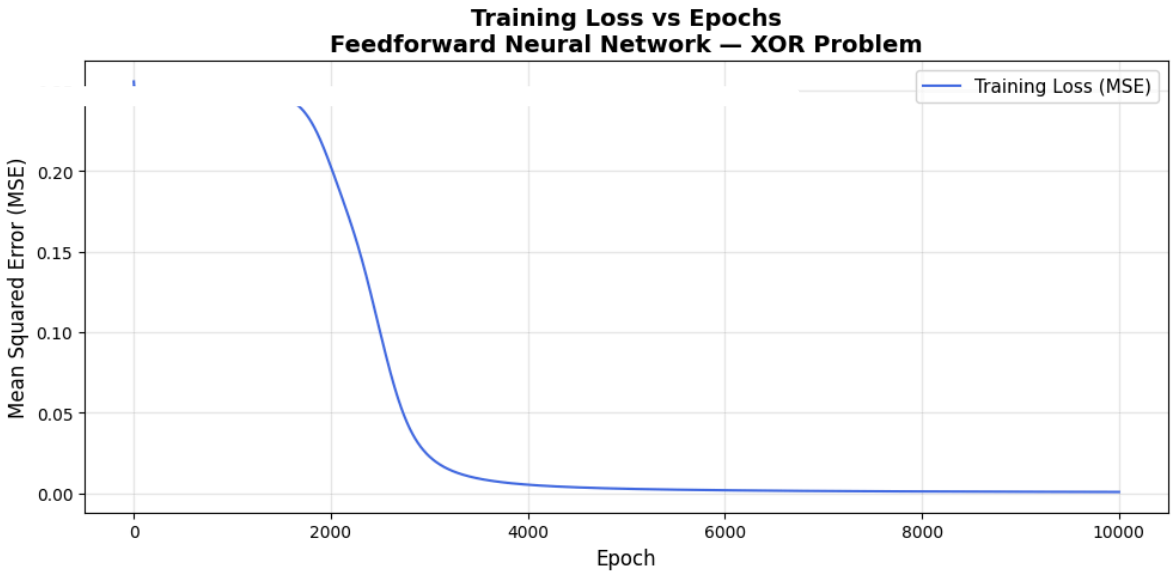
```
header = f"{'':>10}" + "".join(f"{'H'+str(j+1):>12}" for j in range(hidden_s
print(header)
for i in range(input_size):
    row = f"{'X'+str(i+1):>10}" + "".join(f"{W1[i,j]:>12.6f}" for j in range
    print(row)

print("\n[ b1 ] Hidden Layer Biases")
print("-" * 55)
for j in range(hidden_size):
    print(f"  b1[H{j+1}] = {b1[0,j]:>10.6f}")

print("\n[ W2 ] Hidden → Output Layer  (shape: 4×1)")
print("-" * 55)
for i in range(hidden_size):
    print(f"  W2[H{i+1}→Out] = {W2[i,0]:>10.6f}")

print("\n[ b2 ] Output Layer Bias")
print("-" * 55)
print(f"  b2[Out]   = {b2[0,0]:>10.6f}")
print("=" * 60)
```

```
============================================================
         FINAL LEARNED WEIGHTS & BIASES
============================================================

[ W1 ] Input → Hidden Layer  (shape: 2×4)
-------------------------------------------------------
                     H1          H2          H3          H4
        X1    2.833187   -0.067363    3.015014    6.418709
        X2    2.768674   -0.141079    3.090370    6.400821

[ b1 ] Hidden Layer Biases
-------------------------------------------------------
  b1[H1] =  -4.451665
  b1[H2] =   0.160863
  b1[H3] =  -4.821619
  b1[H4] =  -2.857763

[ W2 ] Hidden → Output Layer  (shape: 4×1)
-------------------------------------------------------
  W2[H1→Out] =  -5.964443
  W2[H2→Out] =  -0.853727
  W2[H3→Out] =  -6.337502
  W2[H4→Out] =   9.675636

[ b2 ] Output Layer Bias
-------------------------------------------------------
  b2[Out]   =  -3.427612
============================================================
```

Step 11: Loss Curve Visualization

```
plt.figure(figsize=(10, 5))
plt.plot(range(1, epochs + 1), loss_history, color='royalblue', linewidth=1.
plt.title('Training Loss vs Epochs\nFeedforward Neural Network — XOR Problem
```

```
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Mean Squared Error (MSE)', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Loss at Epoch 1    : {loss_history[0]:.6f}")
print(f"Loss at Epoch {epochs}  : {loss_history[-1]:.6f}")
print(f"Loss Reduction     : {((loss_history[0]-loss_history[-1])/loss_histo
```




```
Loss at Epoch 1    : 0.255675
Loss at Epoch 10000  : 0.000696
Loss Reduction     : 99.73%
```

Summary

Component	Description
Architecture	2 → 4 → 1 (Input → Hidden → Output)
Activation	Sigmoid at all layers
Loss Function	Mean Squared Error (MSE)
Optimizer	Gradient Descent
Learning Rate	0.5
Epochs	10,000
Libraries Used	Only NumPy & Matplotlib

Key Takeaways

1. **Forward Pass** computes predictions layer by layer using weighted sums and activation functions.
2. **Backpropagation** uses the chain rule to compute gradients of the loss w.r.t every weight.
3. **Gradient Descent** updates all weights in the direction that reduces the loss.
4. After training, the network successfully learns the XOR function — a non-linearly separable problem.

Assignment 1 | Name: Shivam Khedkar | PRN: 202301040063 | Batch: 1